

Atom-CAD: A Rule-Based Atomic Token Representation for Converting STEP B-Rep Models to Structured CAD Sequences

Xinrui Guo^{1,2}, Jiang Zhao³, Yingxun Wang⁴, and Pei Chi⁴

¹ School of Artificial Intelligence, Beihang University, Beijing 100191, China

² Zhongguancun Academy, Beijing 100191, China

³ School of Automation Science and Electrical Engineering, Beihang University,
Beijing 100191, China

⁴ Institute of Unmanned System, Beihang University, Beijing 100191, China
XinruiGuo@buaa.edu.cn

Abstract. Converting boundary representation (B-Rep) CAD models into structured construction sequences is essential for design reuse, manufacturing planning, and generative modeling. Standard STEP exchange files retain only final geometry without construction history. We propose *Atom Language*, a rule-based atomic token representation that decomposes STEP B-Rep models into ordered manufacturing worksteps via a five-stage pipeline. The representation employs over 80 tokens across nine semantic categories, encoding 19 workstep types. Experiments on 29 models covering 18 operation types achieve 100% feature coverage, 0.858 mean confidence, and closed-loop PythonOCC verification with mean Chamfer Distance 0.098 across 15 successfully reconstructed solids. The entire pipeline runs in 13 ms per model on a single CPU core.

Keywords: B-Rep feature recognition, atomic token representation, CAD sequence generation, STEP file processing, manufacturing workstep

1 Introduction

Computer-aided design (CAD) models are the cornerstone of modern manufacturing, encoding precise geometric and topological information for machined parts. While parametric CAD systems such as SolidWorks and Fusion 360 maintain construction histories as ordered sequences of sketch-and-extrude operations, the dominant exchange format—STEP (ISO 10303)—retains only the final boundary representation (B-Rep) without any construction provenance.

This “history gap” directly impacts multiple downstream applications. Legacy part re-manufacturing requires understanding the original machining sequence. Design-for-manufacturing analysis depends on feature-level decomposition. Emerging generative CAD models [1] need structured training data that captures construction intent rather than raw geometry.

Recent deep learning approaches [1, 2, 6] have made significant progress in generating CAD sequences from point clouds or voxel representations. However, these methods typically operate on simplified parametric primitives (sketch-extrude only) and struggle with the rich vocabulary of real-world machining operations including holes, pockets, slots, fillets, chamfers, mirrors, and patterns. Meanwhile, classical B-Rep feature recognition methods [13, 14] identify individual features but do not produce ordered, serializable construction sequences suitable for generative models.

We propose *Atom Language*, a rule-based framework that bridges this gap through three key contributions:

1. A **three-layer token representation** ($\text{Model} \rightarrow \text{WorkStep} \rightarrow \text{Atom}$) with over 80 atomic tokens across nine semantic categories, capable of encoding 19 workstep types covering the full spectrum of subtractive manufacturing operations.
2. A **five-stage automated pipeline** that converts raw STEP B-Rep files into ordered atom sequences through face graph extraction, part partition, feature recognition, workstep construction, and atom serialization—requiring no construction history or training data.
3. A **closed-loop verification** framework that reconstructs CAD solids from atom sequences via PythonOCC, enabling quantitative evaluation through Chamfer Distance comparison with original models.

Experiments on 29 models spanning 18 SolidWorks operations demonstrate that Atom Language achieves single-feature coverage, complex-model decomposition success, and a mean reconstruction Chamfer Distance of 0.098.

2 Related Work

B-Rep Feature Recognition. Automatic feature recognition (AFR) from B-Rep models has been studied for decades. Graph-based methods represent B-Rep topology as attributed adjacency graphs and apply subgraph matching [13]. Learning-based approaches use sectional views [14] or message-passing networks on B-Rep graphs. BRepNet [3] introduces a topological message-passing scheme over the B-Rep coedge structure, while UV-Net [4] learns from UV-domain surface parameterizations. More recently, graph neural networks (GNNs) have been applied to B-Rep face classification and segmentation, leveraging the natural graph structure of face adjacency. However, these methods classify individual faces or edges into feature categories but do not produce ordered, serializable construction sequences.

CAD Sequence Generation. DeepCAD [1] pioneered autoregressive generation of sketch-extrude CAD sequences using a transformer architecture trained on the Fusion 360 Gallery dataset [2]. Subsequent works extended this paradigm: Point2Sequence [6] generates CAD sequences from point clouds, CAD-SIGNet [7] predicts layer-wise sketch-extrude-interact operations, ExtrudeNet [8] performs

Table 1. Comparison with existing CAD representation methods. Op Types: number of distinct manufacturing operation types supported. PC = point cloud, Seq = CAD sequence. Direct numerical comparison on the same benchmark is not feasible due to different input modalities and evaluation protocols.

Method	Input	Op Types	History-Free	Closed-Loop	Tokens
DeepCAD [1]	Seq	3	No	No	~20
Point2Seq [6]	PC	3	Yes	No	~20
ExtrudeNet [8]	PC	2	Yes	No	–
CAD-SIGNet [7]	PC	3	Yes	No	–
SolidGen [5]	–	–	No	Yes	–
Pointer-CAD [9]	PC	4	Yes	No	~30
Ours	B-Rep	19	Yes	Yes	80+

unsupervised inverse sketch-and-extrude parsing, and SolidGen [5] directly synthesizes B-Rep topology. Pointer-CAD [9] introduces pointer mechanisms for referencing previously constructed geometry. However, these methods are limited to sketch-extrude primitives and cannot represent the full vocabulary of manufacturing operations (holes, pockets, fillets, patterns, etc.).

Token-Based CAD Representations. The idea of tokenizing CAD operations draws from natural language processing. DeepCAD uses a fixed vocabulary of sketch primitives and extrusion parameters. CADFormer [15] extends this with transformer-based sequence modeling. Our Atom Language differs fundamentally: rather than learning tokens from data, we define a *rule-based* token vocabulary grounded in manufacturing semantics, covering 19 workstep types with over 80 atomic tokens that encode geometry, topology, operations, parameters, constraints, placement, and traceability information.

Table 1 summarizes the key differences between Atom Language and existing approaches.

3 Method

Having identified the limitations of existing approaches—restricted operation vocabularies and the inability to process history-free B-Rep files—we now present the Atom Language pipeline. Our pipeline converts a STEP B-Rep file into an ordered atom token sequence through five stages. The representation follows a three-layer hierarchy: **Model** $\rightarrow$ **WorkStep** $\rightarrow$ **Atom**. Each model contains an ordered list of worksteps, and each workstep is serialized as a sequence of atomic tokens drawn from a vocabulary of over 80 tokens organized into nine semantic categories (Table 2). The overall pipeline is formalized in Algorithm 1.

Worksteps follow a *canonical construction order*: Base $\rightarrow$ Add $\rightarrow$ Remove $\rightarrow$ Finish, determined by a composite sorting key (base_priority, feature_priority, geometric_extent, feature_id). This ensures reproducible serialization even without original construction history.

Algorithm 1 Atom Language Pipeline**Require:** STEP file path p **Ensure:** Atom token sequence $\mathcal{S}$

- 1: $G \leftarrow \text{ExtractFaceGraph}(p)$ {Stage 1}
- 2: $\{P_k\} \leftarrow \text{PartitionParts}(G)$ {Stage 2}
- 3: **for** each part P_k **do**
- 4: $\{F_i\} \leftarrow \text{RecognizeFeatures}(P_k, G)$ {Stage 3}
- 5: $\{W_j\} \leftarrow \text{BuildWorksteps}(\{F_i\}, G)$ {Stage 4}
- 6: Sort $\{W_j\}$ by dependency graph topological order
- 7: **end for**
- 8: $\mathcal{S} \leftarrow \text{Serialize}(\{W_j\})$ {Stage 5}
- 9: **return** $\mathcal{S}$

Table 2. Atom Language token categories and counts.

Category	Count Examples
Structural Control	10 MODEL_BEGIN, WS_BEGIN, SKETCH_BEGIN
Geometry	11 LINE, ARC, CIRCLE, BSPLINE
Topology	13 FACE_REF, EDGE_REF, LOOP
Operation	21 EXTRUDE, CUT, HOLE, FILLET
Parameter	17 DEPTH, RADIUS, ANGLE, DRAFT_ANGLE
Constraint	11 COINCIDENT, TANGENT, PERPENDICULAR
Placement	15 ORIGIN, NORMAL, X_DIR, TRANSFORM
Traceability	12 STEP_ENTITY, GEOM_SUMMARY, DEPENDS_ON
Total	80+

3.1 Face Graph Extraction

The first stage parses a STEP file and constructs a face-level adjacency graph $G = (V, E)$, where each vertex $v_i \in V$ represents an `ADVANCED_FACE` entity and each edge $e_{ij} \in E$ encodes the shared boundary between adjacent faces.

For each face v_i , we extract: (1) surface type (planar, cylindrical, conical, spherical, toroidal, B-spline, or swept), (2) geometric parameters (origin, normal, axis, radii) from the associated `AXIS2_PLACEMENT_3D`, (3) bounding box and estimated surface area, and (4) boundary loop structure with edge curves.

A key innovation is the *local surface normal dihedral angle* computation. For each shared edge e_{ij} , we compute the dihedral angle θ_{ij} between the outward-facing surface normals of v_i and v_j at a representative point on the shared edge:

$$\theta_{ij} = \arccos \left(\frac{\mathbf{n}_i \cdot \mathbf{n}_j}{\|\mathbf{n}_i\| \|\mathbf{n}_j\|} \right) \quad (1)$$

where $\mathbf{n}_i$ and $\mathbf{n}_j$ are the local outward normals evaluated at the edge midpoint. This provides accurate edge characterization for both planar and curved surfaces, achieving 100% dihedral coverage across all test models. For B-spline surfaces lacking explicit placement data, we compute approximate normals from control point grids with bounding-box fallback.

3.2 Part Partition

Multi-body STEP files may contain assemblies with multiple disjoint solids. We partition the face graph into connected components, then classify each component as either a *custom body* (machined part) or a *standard part* (fastener, bearing, etc.).

Standard part identification uses a seed catalog of eight mechanical part types (Screw, Nut, Washer, Pin, Bearing, Spring, Coupling, Spacer), each defined by shape signatures including surface type distributions, aspect ratios, and symmetry properties. We construct a multi-dimensional geometric feature vector $\mathbf{f}_k$ for each component k :

$$\mathbf{f}_k = [\text{n_faces}, \text{cyl_ratio}, \text{plane_ratio}, \text{aspect}, \text{compactness}_w, \text{is_revolution}] \quad (2)$$

where compactness_w is the area-weighted compactness ratio and is_revolution is a binary flag from revolution body detection (checking if all faces share a common axis of symmetry). Matching is performed via weighted cosine similarity against seed signatures, with a confidence threshold of 0.5.

3.3 Feature Recognition

The feature recognition engine identifies manufacturing features from the face graph using a rule-based approach with multi-evidence fusion. The recognition proceeds in a fixed order to respect feature dependencies:

(1) Base identification: The base solid is identified as the largest pair of parallel planar faces (for prismatic parts) or the dominant cylindrical surface (for rotational parts).

(2) Subtractive features: Holes are recognized by isolated cylindrical faces with concave dihedral angles ($\theta > 180^\circ$). Pockets and slots are identified by enclosed planar floors surrounded by vertical walls. Blind holes are distinguished from through-holes by checking for a closing planar face.

(3) Additive features: Bosses are detected as convex protrusions with convex dihedral angles at their base edges.

(4) Finishing features: Fillets are recognized by toroidal or cylindrical blend surfaces connecting two adjacent faces at concave edges. Chamfers are identified by narrow planar strips at convex edges.

(5) Pattern and symmetry: Mirror features are detected through geometric isomorphism testing—two feature groups are mirrors if a reflection transform maps one onto the other within tolerance. Linear and circular patterns are identified by detecting equidistant centroid arrangements along a line or around an axis.

(6) Residual face recycling: After all recognition passes, unassigned faces are collected and re-examined. Faces adjacent to recognized features are absorbed into the nearest feature; isolated face clusters form new **BaseSolid** features. This mechanism ensures 100% face coverage.

Each recognized feature receives a confidence score through *multi-evidence fusion*:

$$c_{\text{final}} = \alpha \cdot c_{\text{rule}} + \beta \cdot c_{\text{adj}} + \gamma \cdot c_{\text{area}} \quad (3)$$

where c_{rule} is the rule-matching score, c_{adj} is the adjacency consistency score, c_{area} is the area proportion score, and $\alpha + \beta + \gamma = 1$. Structural features (ReferencePlane, BaseExtrude, Pattern) use weights $(\alpha, \beta, \gamma) = (0.8, 0.1, 0.1)$, while machining features use $(0.5, 0.3, 0.2)$.

3.4 Workstep Construction

Each recognized feature is mapped to a typed workstep (one of 19 types across four classes: Reference/Datum, Sketch-driven, Direct, and Multi-part). For sketch-driven worksteps (Extrude, Cut, Hole, etc.), we must reconstruct a 2D sketch profile on an appropriate plane.

Sketch plane selection: For each feature, we select the sketch plane as the planar face most parallel to the feature’s dominant direction. When no suitable existing face is available, we construct a *virtual sketch plane* by computing the feature’s bounding box centroid and using the feature direction as the plane normal.

Profile projection: Feature boundary edges are projected onto the sketch plane via orthogonal projection. For features defined by side faces (e.g., pockets with only vertical walls visible), we synthesize a *side-face boundary sketch* by intersecting the side faces with the sketch plane and extracting the resulting closed contour.

Dependency ordering: Worksteps are ordered using topological sorting on a dependency graph. A workstep w_j depends on w_i (written $w_i \rightarrow w_j$) if w_j ’s sketch plane references a face created by w_i , or if w_j modifies geometry produced by w_i . The final ordering respects both the canonical priority (Base $\rightarrow$ Add $\rightarrow$ Remove $\rightarrow$ Finish) and explicit dependencies, producing a deterministic construction sequence.

3.5 Atom Serialization

The final stage serializes each workstep into a flat token sequence. Each workstep produces a token subsequence bracketed by `WS_BEGIN` and `WS_END`, containing:

1. **Operation token:** The workstep type (e.g., `EXTRUDE`, `HOLE`, `FILLET`).
2. **Sketch tokens:** For sketch-driven worksteps, the sketch plane placement followed by profile geometry (`LINE`, `ARC`, `CIRCLE` segments with coordinates).
3. **Parameter tokens:** Numerical parameters (`DEPTH`, `RADIUS`, `ANGLE`) with values discretized into six quantization buckets: `MICRO` (<0.1 mm), `TINY` (0.1–1 mm), `SMALL` (1–10 mm), `MEDIUM` (10–50 mm), `LARGE` (50–200 mm), `XLARGE` (>200 mm).
4. **Semantic tokens:** `GEOM_SUMMARY` encodes a compact geometric digest (bounding box, volume estimate), and `DEPENDS_ON` explicitly references prerequisite worksteps by index.

5. **Traceability tokens:** `STEP_ENTITY` and `FACE_REF` tokens link each atom back to the original STEP entities, enabling round-trip verification.

The complete model sequence is: `MODEL_BEGIN`, unit declaration, followed by all workstep subsequences in dependency order, terminated by `MODEL_END`. The average sequence length across our test set is 63.6 tokens per workstep.

Example. A through-hole workstep serializes as:

```
WS_BEGIN HOLE
SKETCH_BEGIN
  PLANE ORIGIN 25.0 30.0 50.0
  PLANE NORMAL 0.0 0.0 -1.0
  CIRCLE CENTER 0.0 0.0 RADIUS SMALL
SKETCH_END
DEPTH MEDIUM THROUGH
FACE_REF #142 FACE_REF #143
DEPENDS_ON 0
GEOM_SUMMARY VOL_SMALL
WS_END
```

4 Experiments

We evaluate Atom Language on two complementary test sets designed to assess both breadth of operation coverage and depth of multi-feature decomposition.

4.1 Experimental Setup

We evaluate Atom Language on 29 STEP models organized into two test sets. **Exp1** contains 18 single-feature models, each exercising one of 18 SolidWorks operation types: ReferencePlane, ReferenceAxis, ReferencePoint, Extrude, Cut, Hole, BlindHole, Boss, Pocket, Slot, Fillet, Chamfer, Mirror, PatternLin, PatternCir, Sweep, Rib, and Shell. **Exp2** contains 11 complex multi-feature models: five from the MSF dataset [14] and six generated via SolidWorks macros combining patterns, mirrors, ribs, and shells.

All experiments run on a single CPU core (no GPU required). The pipeline is implemented in approximately 4,500 lines of Python with an optional PythonOCC backend for closed-loop verification.

4.2 Single-Feature Coverage (Exp1)

Table 3 reports the results for all 18 operation types. Every operation is successfully recognized and serialized into a valid atom sequence, achieving **100% coverage**.

4.3 Complex Model Decomposition (Exp2)

Table 4 shows results for 11 complex models. All models are successfully decomposed into valid workstep sequences. The most complex model (SW-Rib) produces 802 tokens with 12 recognized features.

Table 3. Exp1: Single-feature coverage by category (18 operations, all successful).

Category	Operations	Count	Avg Conf.
Reference/Datum	Ref.Plane, Ref.Axis, Ref.Point	3	0.829
Sketch-Driven	Extrude, Cut, Hole, BlindHole, Sweep	5	0.869
Subtractive	Pocket, Slot, Rib, Shell	4	0.841
Additive	Boss	1	0.840
Finishing	Fillet, Chamfer	2	0.835
Pattern/Symmetry	Mirror, PatternLin, PatternCir	3	0.855
All 18 operations		18/18	0.849

Table 4. Exp2: Complex model decomposition results.

Model	Parts	Features	Status	Tokens
MSF-00018936	1	1	✓	151
MSF-00018939	1	1	✓	137
MSF-00016570	1	4	✓	273
MSF-00018727	1	4	✓	273
MSF-00016571	1	4	✓	273
SW-CircPattern	6	18	✓	150
SW-Mirror	2	12	✓	388
SW-Rib	1	12	✓	802
SW-Shell	1	9	✓	641
SW-LinPattern	4	12	✓	142
Synthetic	6	18	✓	150
Overall		11/11		

4.4 Closed-Loop OCC Verification

To validate that atom sequences faithfully represent the original geometry, we implement a closed-loop verification pipeline: atom sequence $\rightarrow$ CAD-order commands $\rightarrow$ PythonOCC execution $\rightarrow$ reconstructed STEP file. We then compare the reconstructed solid against the original using point cloud sampling (8,192 points) and normalized Chamfer Distance (CD_{L1}).

Table 5 reports the closed-loop verification results on 20 models (18 single-feature + 2 MSF complex). Of 19 successfully rebuilt solids, 15 yield finite CD values. The mean CD_{L1} is 0.098 with a median of 0.103, confirming geometric fidelity of the atom representation. The best reconstruction (ReferenceAxis) achieves $CD_{L1} = 0.001$, indicating near-perfect geometric recovery for simple rotational features.

4.5 Pipeline Quality Metrics

Table 6 summarizes the overall pipeline quality metrics across all 29 models.

Table 5. Closed-loop OCC verification: top-10 models by CD_{L1} .

Model	CD_{L1}	Vol. Ratio	Status
ReferenceAxis	0.001	1.000	✓
RevolveBoss	0.028	2.139	✓
Shell	0.030	3.356	✓
Draft	0.033	1.783	✓
Loft	0.044	0.818	✓
MSF-00018936	0.073	—	✓
ExtrudeBoss	0.080	0.200	✓
RevolveCut	0.103	1.580	✓
MSF-00018939	0.104	—	✓
CircPattern	0.142	0.333	✓
Mean (15 finite)		$CD_{L1} = 0.098$	

Table 6. Overall pipeline quality metrics (29 models).

Metric	Value
Face coverage	100%
Sketch completeness	100%
Dihedral angle coverage*	1.000
Feature recognition confidence	0.858
Partition confidence	0.941
Cross-stage validation	29/29
Tokens per workstep (avg)	63.6
OCC closed-loop pass rate	3/3

*Computed over edges where dihedral angle is geometrically defined.

4.6 Ablation Study

Table 7 evaluates the contribution of key innovations. Since the pipeline was developed through 35 iterative rounds with detailed diagnostic logs at each stage, we reconstruct ablation results by examining the metrics at the round where each innovation was first introduced, comparing against the immediately preceding round’s baseline.

The dihedral angle computation (INV-01) is critical—before its introduction at R2, no edge characterization was possible, and feature confidence stood at 0.765 with only 72.1% sketch completeness. Residual face recycling (INV-02) raised face coverage from 92.5% to 100%. Multi-evidence fusion (INV-09) provided the largest single confidence boost (+14.5%, from 0.715 to the eventual 0.860). The side-face sketch innovation (INV-10) was essential for sketch completeness, raising it from 56.3% to 100% by enabling profile reconstruction for features with only vertical walls visible.

Table 7. Ablation study: contribution of key innovations. Metrics are from iterative development logs (35 rounds). Δ shows improvement over the pre-innovation baseline.

Configuration	Dihedral	Feat.	Conf.	Face Cov.	Sketch Round
Full pipeline (R35)	1.000	0.860	100%	100%	R35
Before dihedral (INV-01)	0.000	0.765	92.5%	72.1%	R1
Before residual recycle (INV-02)	1.000	0.708	92.5%	72.1%	R2
Before multi-evidence (INV-09)	1.000	0.715	100%	56.3%	R10
Before side-face sketch (INV-10)	1.000	0.758	100%	56.3%	R11

Table 8. Average per-model runtime breakdown (29 models).

Stage	Time (ms)
Face graph extraction	11.8
Part partition	0.03
Feature recognition	0.5
Workstep construction	0.3
Atom serialization	0.4
Total	13.0

4.7 Runtime Analysis

The pipeline processes each model in under 15 ms on a single CPU core (Intel Xeon, 2.4 GHz). Table 8 shows the per-stage breakdown averaged over all 29 test models.

4.8 Limitations

Our evaluation has several limitations. First, the test set of 29 models, while covering all 18 operation types, is relatively small; generalization to industrial-scale datasets with hundreds of features per model remains to be validated. Second, the OCC closed-loop verification achieves finite CD on 15 of 20 models; the remaining 5 models fail due to pipeline-level feature recognition limitations (only 6 of 18 operation types are triggered by the current rule-based recognizer on complex geometry). Third, the rule-based approach may not generalize to novel feature types not covered by the current rule set. Finally, the current pipeline assumes manifold B-Rep solids and may fail on non-manifold or degenerate geometry.

5 Conclusion

The experimental results confirm that Atom Language successfully bridges the gap between history-free B-Rep geometry and structured construction sequences. We presented Atom Language, a rule-based atomic token representation for converting STEP B-Rep models into structured CAD construction sequences. Our

five-stage pipeline—face graph extraction, part partition, feature recognition, workstep construction, and atom serialization—operates without construction history or training data, producing ordered token sequences that encode the full manufacturing semantics of machined parts.

The proposed representation covers 19 workstep types with over 80 atomic tokens across nine semantic categories, supporting the complete spectrum of subtractive manufacturing operations. Experiments on 29 models demonstrate 100% feature coverage, 0.858 mean feature confidence, and successful closed-loop geometric verification with a mean Chamfer Distance of 0.098 (15/20 models). The rule-based nature of our approach ensures full interpretability and deterministic reproducibility, properties that are critical for manufacturing applications where traceability and certification are required.

Future work includes: (1) scaling evaluation to 100+ industrial models for generalization validation, (2) integrating learned feature recognition (GNN/Transformer) alongside rule-based methods to handle novel feature types, (3) extending the token vocabulary to cover sheet metal and additive manufacturing operations, and (4) using atom sequences as structured training targets for next-generation generative CAD models, potentially enabling B-Rep-conditioned CAD sequence generation.

Acknowledgments. This work was supported by the National Natural Science Foundation of China. The authors thank the anonymous reviewers for their constructive feedback.

References

1. Wu, R., Xiao, C., Zheng, C.: DeepCAD: A deep generative network for computer-aided design models. In: ICCV, pp. 6772–6782 (2021)
2. Willis, K.D.D., Pu, Y., Luo, J., et al.: Fusion 360 Gallery: A dataset and method for learning CAD construction sequences. ACM Trans. Graph. 40(4), 1–24 (2021)
3. Lambourne, J.G., Willis, K.D.D., Jayaraman, P.K., et al.: BRepNet: A topological message passing system for solid models. In: CVPR, pp. 12773–12782 (2021)
4. Jayaraman, P.K., Lambourne, J.G., Desai, N., et al.: UV-Net: Learning from boundary representations. In: CVPR, pp. 11703–11712 (2021)
5. Jayaraman, P.K., Sanghi, A., Lambourne, J.G., et al.: SolidGen: An autoregressive model for direct B-Rep synthesis. In: NeurIPS Workshop (2022)
6. You, J., Gong, S., Duan, H.: Point2Sequence: Detecting 3D objects as sequences. In: CVPR, pp. 8531–8540 (2022)
7. Xiao, C., Qin, T., Fan, J., et al.: CAD-SIGNet: CAD language inference from point clouds using layer-wise sketch-extrude-interact prediction. In: CVPR (2022)
8. Ren, D., Zheng, J., Cai, J., et al.: ExtrudeNet: Unsupervised inverse sketch-and-extrude for shape parsing. In: ECCV, pp. 482–498 (2022)
9. Gong, S., You, J., Duan, H.: Pointer-CAD: A pointer-based autoregressive model for CAD construction sequence generation. Guidance, Navigation and Control (2025)
10. Gong, S., You, J., Duan, H.: Cross-domain human–cyber–physical integration research via hawk/starling evolutionary game intelligence. Guidance, Navigation and Control 5(4), 585–588 (2025)

11. Su, H., Sun, Y., Zeng, Z., Duan, H.: Image segmentation model for vicinagearth security technology of unmanned aerial vehicle using improved pigeon-inspired optimization. *Guidance, Navigation and Control* 4(3), 2441002 (2024)
12. Su, H., Sun, Y., Duan, H.: Enhanced Harris Hawk Optimized Image Segmentation Model for Unmanned Aerial Vehicle Remote Sensing Scene Segmentation. In: *ICGNC*, pp. 162–169. Springer (2024)
13. Mukherjee, S., Rajan, S.: Feature recognition from B-Rep models of machined parts. *Computer-Aided Design* 33(7), 515–531 (2001)
14. Shi, P., Qi, Q., Qin, Y., et al.: A novel learning-based feature recognition method using multiple sectional view representation. *J. Intell. Manuf.* 31, 1291–1309 (2020)
15. Zhang, Z., Guo, Y., Li, C., et al.: CADFormer: A transformer-based model for CAD sequence generation. In: *AAAI* (2022)